Predicting Goodreads Review Popularity from Text and Reviewer Features

Project Write-up

1. Problem Statement

Online book reviews vary widely in how much engagement they receive from other readers. Predicting whether a review will become “popular” (i.e. attract significant reader engagement) has practical value for platforms like Goodreads, which could surface high-quality or high-engagement reviews earlier, and for authors/publishers seeking to understand what drives reader response. This project frames popularity prediction as a binary classification task: given a review's text and metadata about the reviewer and the book, predict whether the review is popular or not. We specifically investigate whether combining semantic text embeddings with simpler engineered features (review statistics and reviewer activity history) outperforms either feature type alone, and use explainability analysis to understand which signals actually drive the model's predictions.

2. Dataset

We used the public Goodreads book reviews dataset (goodreads_reviews_dedup.json.gz), working with a sample of 200,000 reviews. Each review includes the raw review text, a star rating, a timestamp, and identifiers linking the review to its reviewer and book. From this raw data we engineered two feature groups: a **non-text** group (reviewer's total review count, days since the review was posted, the review's star rating, and the rating's difference from the book's average rating) and a **text-statistics** group (word count, average sentence length, average word length, presence of quoted text, exclamation mark count, question mark count, and sentiment polarity). The target label (“popular”) was derived from the dataset's engagement signal. Reviews were split 70/15/15 into train/validation/test sets, **grouped by book_id** so that reviews of the same book never appear across splits (avoiding leakage), yielding 54,434 / 29,813 / 28,414 reviews respectively. The training split was class-balanced via random undersampling of the majority class; validation and test sets were left at natural class balance to give realistic evaluation metrics.

3. Approach

We compared three feature representations and two model families to isolate the contribution of semantic text understanding versus simpler engineered signals:

- **TF-IDF** (5,000-term vocabulary) as a classical text-only baseline, with logistic regression.
- **MiniLM sentence embeddings** (all-MiniLM-L6-v2, 384-dim) as a text-only semantic baseline, with both logistic regression and XGBoost.
- **Engineered features (B) + MiniLM embeddings** concatenated, with both logistic regression and XGBoost, to test whether reviewer/book metadata adds signal beyond what the text embedding already captures.

Logistic regression hyperparameters (regularization strength C) were tuned via validation-set ROC-AUC over a small grid. XGBoost was trained with early stopping on the validation set (up to 1,000 trees, max depth 6, subsample/colsample 0.8). Sentence embeddings were computed on CPU in resumable chunks after a GPU run caused system instability. After selecting the best-performing model, we applied **SHAP (TreeExplainer)** to the held-out test set to quantify and visualize feature importance, grouping the 384 individual embedding dimensions into a single aggregate block alongside the 11 named engineered features for interpretability. We also built an interactive Streamlit demo that accepts a pasted review and reviewer/book context, and returns a live prediction with a SHAP-based explanation.

4. Implementation Overview

The pipeline is organized as a small set of scripts: **build_features.py** and **prepare_data.py** construct the engineered feature table and labels from the raw Goodreads data; **embedding_models.py** computes (and caches to disk) MiniLM embeddings, trains and evaluates all five model/feature-set combinations, and persists the best model and its feature

scaler; **step7_shap.py** loads the saved model and reproduces the test split to compute grouped SHAP importances and summary plots; and **step8_streamlit_demo.py** wraps the saved model and SHAP explainer in an interactive web demo. All code and a setup README are in the project's GitHub repository.

5. Results

Features	Model	Accuracy	F1	ROC-AUC	PR-AUC
TF-IDF	LogReg	0.642	0.430	0.694	0.370
MiniLM embeddings	LogReg	0.593	0.396	0.651	0.311
MiniLM embeddings	XGBoost	0.585	0.403	0.655	0.315
B + MiniLM embeddings	LogReg	0.646	0.426	0.687	0.351
B + MiniLM embeddings	**XGBoost**	**0.657**	**0.457**	**0.733**	**0.417**

The combined feature set with XGBoost performed best on every metric (ROC-AUC 0.733, PR-AUC 0.417, F1 0.457), confirming that engineered reviewer/book metadata adds real predictive signal beyond what the text embedding alone captures. Notably, TF-IDF alone (ROC-AUC 0.694) was competitive with MiniLM embeddings alone (ROC-AUC 0.651) — at this scale, simple word-frequency features held their own against semantic embeddings used in isolation.

SHAP analysis of the best model showed the aggregated embedding block contributing more to predictions than any single engineered feature, with review length (num_words), reviewer activity (user_reviews), and rating deviation (rating_diff) the most influential engineered features — stylistic features (sentiment, punctuation counts, presence of quotes) contributed comparatively little. Directionally, longer reviews and more active reviewers were associated with higher predicted popularity, while lower star ratings showed a mild positive association with popularity, a counter-intuitive pattern suggesting critical reviews may attract more reader engagement.

6. Conclusions

Combining semantic text embeddings with simple engineered reviewer/book features, scored with a gradient-boosted tree model, gave the strongest and most balanced performance of all configurations tested. Absolute performance (ROC-AUC 0.73) indicates the task carries genuine but moderate predictive signal from this feature set — review popularity is evidently influenced by factors beyond what text and basic metadata capture, such as book-level or platform-level dynamics not modeled here. The explainability analysis provides an interpretable account of the model's behavior and surfaces at least one actionable, non-obvious finding (the rating/popularity relationship) worth further investigation. Future work could explore book- or author-level features, richer sentiment/style modeling, and addressing the moderate PR-AUC with more targeted handling of class imbalance at evaluation time.